TusCore Fleet Manager — Technical Report

Oct 25, 2026 · @asad

Summary

TusCore now runs a simulated warehouse in which 12 robots of four types, from four vendors speaking four different protocols, work as one fleet under one fleet manager. All four phases of the 8-week plan are complete, and the system has been tested on three warehouse layouts with 12, 19 and 40 robots.

The brief asks the system to answer four questions at all times. It does:

Question	How TusCore answers it
How do I talk to each robot?	One adapter per vendor translates TusCore commands into VDA 5050 over MQTT, REST, TCP/JSON or MAVLink-style messages, and the robots' replies back.
Which robot takes this task?	Capability-based allocation, using a greedy method and the Hungarian algorithm. Orders that need two robot types are dispatched together.
Which path does it follow?	A* route planning on the grid, with lane rules so opposing traffic uses different lanes.
How do robots avoid blocking each other?	Every robot reserves the next cell before entering it, deadlocks are detected and resolved automatically, and pallet stackers have right of way.

Headline result (busy day, 10 full 8-hour shifts per scenario, identical orders and faults in both): compared with one separate fleet manager per vendor, TusCore cuts average order cycle time by 22% (219 s to 170 s), waiting time by 16%, robot time lost in traffic by 79%, and operator interventions from 46 per shift to 0. There were 0 collisions and 0 permanent deadlocks in every test shift.

All figures come from the simulation and should be validated on a real site.

Scenario and requirement coverage

Every "done" check in the 8-week plan is met. The robot fleet follows the brief exactly; the numbers are starting values and can be changed in one configuration file.

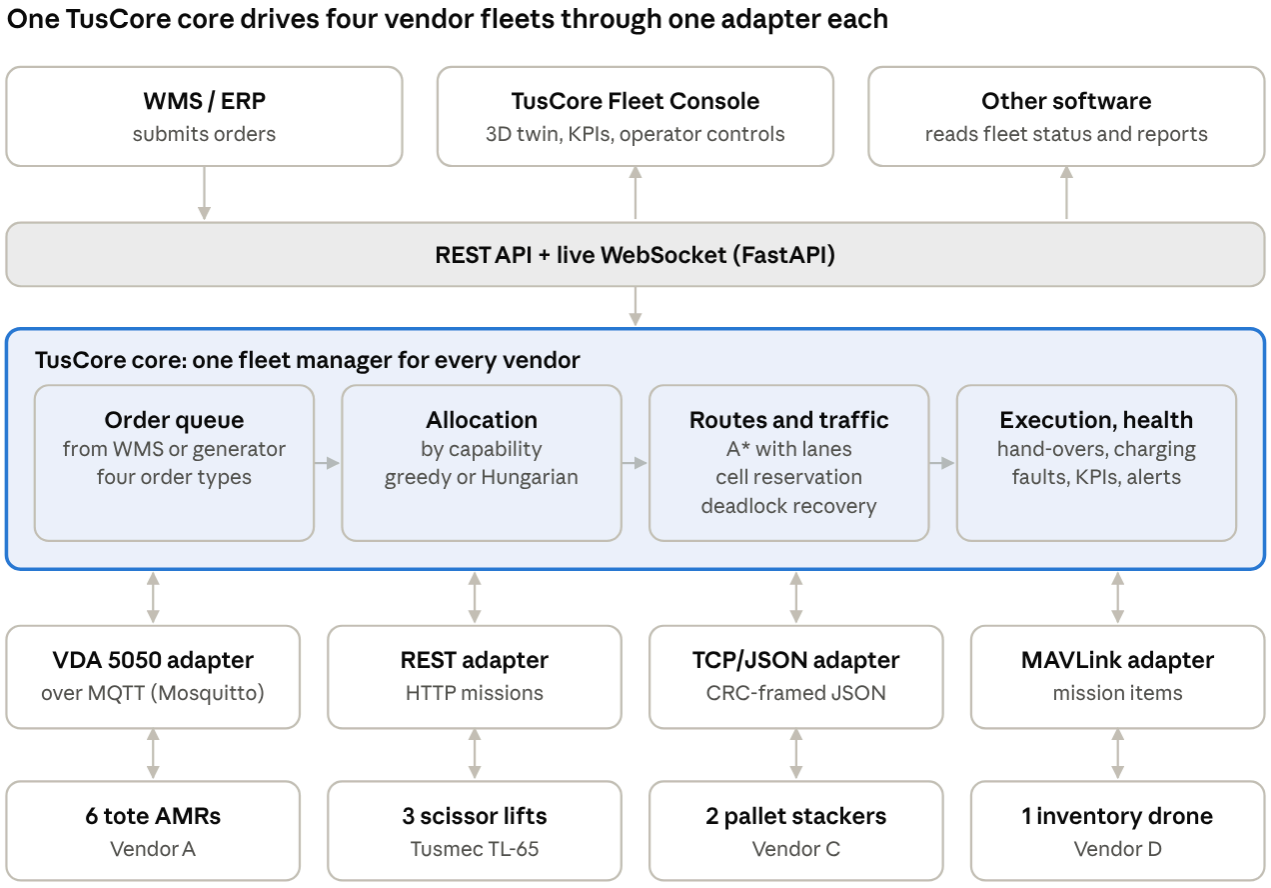
Robot	Count	Vendor	Interface	Speed (cells/s)	Job
Tote AMR	6	Vendor A	VDA 5050 over MQTT	1.0	Carries boxes from shelves to packing
Scissor-lift AMR (Tusmec TL-65)	3	Tusmec	REST API	0.7	Picks from high shelf levels, hands over to a tote AMR
Pallet stacker AGV	2	Vendor C	Custom TCP/JSON	0.5	Moves pallets from the pallet area to the loading dock
Inventory drone	1	Vendor D	MAVLink-style	2.0	Flies over shelf rows, counts stock, flags mismatches

Phase	Done check in the brief	Status
Weeks 1-2: the twin comes alive	Browser shows all 12 robots driving to targets you click	Met. Python simulation core, A* path planning, layout loaded from YAML, live 3D and 2D view
Weeks 3-4: right robot for the job	An 8-hour shift runs automatically and every order type is completed	Met. Four order types, greedy and Hungarian allocation, lift-to-tote hand-over
Weeks 5-6: sharing the floor safely	Full shift with 0 collisions, 0 permanent deadlocks, fault recovery working	Met. Cell reservation, deadlock recovery, priority rules, staggered charging, fault reassignment, vendor adapters
Weeks 7-8: dashboard, comparison, handover	Customer-ready demo, video and technical report	Software met: live dashboard, KPIs, comparison (10 runs each), what-if tool, REST API with docs. This report is the technical report; the demo video is recorded separately

Also delivered: a Git repository with README, a one-command Docker setup that starts a real Mosquitto MQTT broker, auto-generated API documentation, and automated tests (13 passing).

Architecture

One core runs every robot; vendors differ only in their adapter. Orders enter through the REST API, flow through the core from left to right, and leave as each vendor's own protocol messages, while status messages come back the same way.



TusCore architecture · 4 layers, 4 vendor fleets

The arrows between adapters and robots run both ways: commands go down, status comes back up.

Part	Files	Role
Layout and fleet	`config/*.yaml`	Map, zones, robot types and counts, order rates, battery, fault and traffic settings. A new layout needs no code change
Simulation and fleet manager	`sim.py`, `robots.py`, `tasks.py`, `orders.py`	Task execution, charging, faults, KPIs, the separate-managers comparison mode
Allocation	`allocator.py`	Greedy and Hungarian

Part	Files	Role
Routes and traffic	`planner.py`, `traffic.py`, `grid.py`	A* with lanes, cell reservation, deadlock detection, aisle closures
Vendor adapters	`adapters/`	VDA 5050, REST, TCP/JSON, MAVLink-style
Experiments	`experiments.py`	Comparison and what-if runs in parallel on all CPU cores
API and MQTT	`main.py`, `mqtt_bridge.py`	REST, WebSocket, console hosting, mirror of VDA 5050 traffic to Mosquitto
Fleet Console	`frontend/`	3D twin (Three.js), 2D plan, dashboard pages, charts

Task allocation

Orders ask for a capability, never for a vendor, so any robot that can do the job is a candidate. Each robot type declares what it can do in the layout file: `carry_tote`, `high_pick`, `move_pallet` or `scan`.

Orders. Four order types arrive over the shift as independent random (Poisson) streams with a fixed seed, so every run can be repeated exactly. Orders can also be submitted from the console or by any other system through `POST /api/orders`.

Order	Robots it needs	Steps
Box pick	1 tote AMR	Pick the box at the shelf, drop it at a packing station
High-shelf pick	1 scissor lift + 1 tote AMR	Lift picks from the top level; tote meets it in the aisle; hand-over; tote delivers to packing
Pallet move	1 pallet stacker	Lift the pallet in the pallet area, set it down at the loading dock
Inventory scan	1 drone	Fly along a shelf row, flag a stock mismatch if found

Cost of an assignment = travel time from the robot to the order's first stop (distance divided by robot speed).

- **Greedy (baseline):** take the oldest order and give it the nearest free robot that can do it.
- **Hungarian (improved):** for each capability, solve the best overall match of waiting orders to free robots, minimising total travel time, using `scipy.optimize.linear_sum_assignment`. An ageing bonus (0.5 s of cost removed per second waited) stops far-away orders from waiting forever.

Two-robot orders. A high-shelf pick starts only when both a lift and a tote are free. Both are dispatched at once: the lift picks while the tote drives to the aisle cell next to it, whoever arrives first waits, and the hand-over starts when both are there. An order only starts if its work cells (pick, hand-over, drop) are not already reserved by another active order, which prevents two orders fighting over the same shelf bay.

Finding. With real traffic the two methods differ by only a few percent, because the bottleneck is congestion and robot count rather than assignment. Hungarian is the default.

Traffic management and reliability

Collisions are impossible by design: a ground robot owns the cell it stands on and must reserve the next cell before it leaves. A collision counter checks this every step; it has stayed at 0 in every test.

Route planning. A* on the grid with 4-direction moves. Lane discipline adds a small cost for driving in the wrong lane (eastbound on even rows, westbound on odd rows, and the same for columns), so opposing robots naturally use different lanes. This alone cut deadlocks from about 1,300 to about 150 per shift. The drone flies over the racks and is not part of ground traffic.

Priority. Pallet stacker (3) before scissor lift (2) before tote AMR (1). Higher priority wins a contested cell, and the lowest priority gives way in a deadlock, so pallets go first in narrow aisles.

Deadlock handling. TusCore keeps a wait-for graph: each waiting robot points at the robot whose cell it needs, or whose hand-over it is waiting for. A loop in that graph is a deadlock. The recovery ladder:

1. The robot that holds the contested cell and has the lowest priority takes a detour or side-steps.
2. An idle robot standing in someone's way is told to make way.
3. A robot blocked by a robot that is standing still re-plans around it.
4. If the same deadlock lasts 10 s, the newest order is put back in the queue.

5. Safety net: a robot stuck for 300 s is marked faulted. This never triggered in the final stress tests.

Charging. Below 12% a robot charges at once. Below 30% it charges only if at most half of its robot type is already charging. Below 50% it tops up only when no order is waiting for its skills. Robots leave the charger at 90%, and charger bays are limited.

Faults. Robots fail at random (on average every 6 hours per robot) or on demand from the console. A failed robot stops and blocks its cell; its order and any partner robot are released, the order goes back to the front of the queue, and the robot is repaired after 120 s.

Operator controls. Pause the whole fleet, close an aisle for maintenance (routes avoid it and orders located there wait), send a robot to a charger, live alerts, and a congestion heatmap showing where robots lose time waiting.

Verified. Full 8-hour shifts across 3 layouts, 2 allocation methods and many random seeds, with about 14 robot faults per shift: every order completed, 0 collisions, 0 permanent deadlocks.

Vendor adapter layer

The fleet manager knows one robot model (position, state, battery, capabilities) and gives one kind of command (a route plus an action). Each vendor's adapter translates that into the vendor's own messages and decodes the robot's status replies. Every command to every robot goes through its adapter, and every message is shown live in the console's Protocol tab.

Vendor	Adapter	Command sent	Status received
Vendor A (tote AMR)	VDA 5050 v2.0 over MQTT	`order` on `uagv/v2/<maker>/<serial>/order`: nodes and edges, action on the last node	`state`: position, driving, battery, errors
Tusmec (scissor lift)	REST API	`POST /api/v1/robots/{id}/missions` with waypoints, answered 202 Accepted	`GET .../status`: pose, battery, status
Vendor C (pallet stacker)	TCP/JSON	One JSON line per command (`MOVE_PATH`), protected by a CRC32 checksum	`STATUS` frame: position, charge, mode

Vendor	Adapter	Command sent	Status received
Vendor D (drone)	MAVLink-style	`MISSION_COUNT` + `MISSION_ITEM_INT` per waypoint, `MISSION_ACK` back	`HEARTBEAT`, `LOCAL_POSITION_NED`, `BATTERY_STATUS`

Real MQTT. When the system runs with Docker, every VDA 5050 message is also published on a real Mosquitto broker under its standard topic, and can be watched with `mosquitto_sub -t "uagv/#" -v`. This was verified against a live broker.

Adding a vendor means writing one adapter class with four methods (encode and decode orders, encode and decode state), adding one line to the adapter registry, and setting `adapter:` for that robot type in the layout file. Nothing else in the system changes. Automated tests check that all four adapters translate commands and status both ways without loss.

Vendor outage. A vendor can be taken offline from the Vendors panel. Its robots finish their current mission, no new missions are sent to them, an alert is raised, and the rest of the fleet keeps working.

TusCore Fleet Console

The console is a browser app in TusCore branding: logo top-left, "Powered by Tusmec" in the footer, navy #0A1A3C and cyan #2E9FE8, dark theme by default. It works offline, because the 3D and chart libraries are bundled with it.

Page	What it shows
Live	3D digital twin (or 2D plan) with KPI cards (orders, throughput, wait, utilization, energy, on-time %) and a safety strip (collisions, deadlocks, faults, traffic delay). Side panel: robot cards, order center with manual order form, vendor panel, event log, protocol inspector
Overview	The client view: each vendor's fleet at a glance, how TusCore runs the site as five live steps, what needs attention now, and the proven value from the comparison
Analytics	Throughput and queue over the shift, utilization per robot type, battery and faults, completed orders by type, a per-robot table, CSV export and a printable shift report

Page	What it shows
Compare	Separate managers vs TusCore, 10 shifts each, with averages, spread and per-run chart
What-if	Change robot counts, order volume, chargers or layout and see the KPI impact, with a return-on-investment and payback estimate
API	Every endpoint, MQTT status, how to submit orders from a WMS, how to add a vendor

The 3D twin shows the real work. Each robot type has its own animated model. Cartons leave their real shelf slot and go into the tote's bin, then onto the packing table and conveyor. The scissor lift raises its platform to the top level and hands the carton to a tote. Pallets wait where they will be picked up, ride on the forks and stack at the dock until the truck takes them. The drone takes off and scans the racks. A floating vendor marker keeps every robot visible; a red beacon means fault and an amber one means waiting in traffic.

Operator controls on the twin: Pause fleet, Maintenance (close or re-open an aisle), Heatmap, and Send to charger on any idle robot's card. The mode switch in the header changes the live twin between one TusCore manager and separate vendor managers.

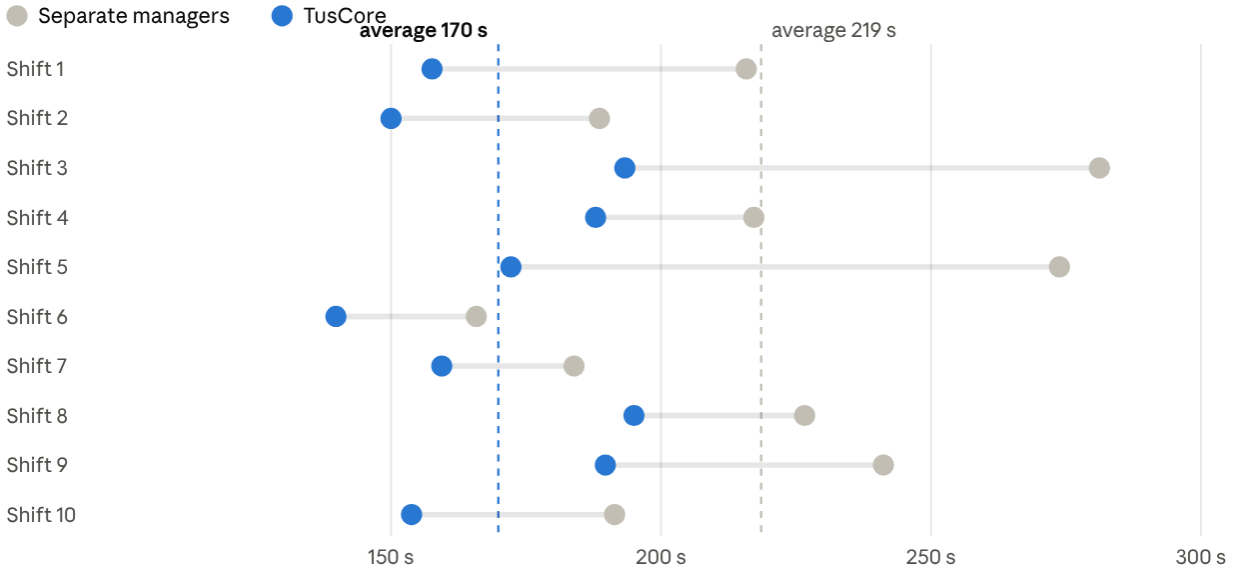
REST API. 32 endpoints with interactive documentation at `/docs`, so a WMS or ERP can submit orders and read fleet status, KPIs, events and reports. A WebSocket pushes the live fleet state 10 times a second.

Week 7-8 results: separate managers vs TusCore

TusCore was faster in every busy-day shift, cutting average order cycle time from 219 s to 170 s (-22%). Scenario A runs each vendor's robots under its own fleet manager, which is today's typical setup. Scenario B coordinates all four vendors with TusCore. Both run the same orders and the same robot faults (identical random seeds) for full 8-hour shifts; only the coordination differs.

TusCore finished orders faster in all 10 busy-day shifts

Average order cycle time per 8-hour shift, seconds; same orders and faults in both scenarios



results/comparison_busy_load.json · 10 shifts per scenario, order volume x1.15

The gap comes from what separate managers cannot see: deadlocks between two vendors' robots wait for an operator, another vendor's parked robot cannot be asked to move, and a high-shelf pick becomes two separate jobs with a hand-over delay.

Scenario (8 h shifts)	Runs per side	Cycle time	Wait before start	Robot time lost to traffic	Operator interventions per shift	Collisions
Normal day, 12 robots	10	124 s to 107 s (-14%)	24.9 s to 24.3 s (-2%)	168 to 38 min (-78%)	36 to 0	0
Busy day (+15%), 12 robots	10	219 s to 170 s (-22%)	104 s to 87 s (-16%)	214 to 44 min (-79%)	46 to 0	0
Peak season (+30%), 12 robots	10	1,407 s to 1,156 s (-18%)	1,144 s to 1,074 s (-6%)	237 to 49 min (-79%)	51 to 0	0
Regional DC, 19 robots	3	131 s to 117 s (-11%)	10.1 s to 11.0 s (+9%)	167 to 47 min (-72%)	28 to 0	0
Mega DC, 40 robots	2	167 s to 148 s (-12%)	12.4 s to 12.4 s (0%)	566 to 134 min (-76%)	94 to 0	0

Orders completed per shift were equal within 2% in every scenario, because orders arrived slower than the fleet could handle them; at peak season both setups saturate and TusCore completed 1.7% more. Every order was completed in every run, and there were 0 permanent deadlocks.

What-if example. At peak volume (+30%), adding two tote robots (6 to 8) raised orders per 4-hour shift from 1,229 to 1,298 (+5.6%) and cut average cycle time from 509 s to 102 s (-80%). The console turns such results into a payback estimate using the client's own robot costs.

Scale. On the 40-robot layout a full 8-hour shift completes about 4,100 to 4,250 orders, with one TusCore core running all 40 robots.

Limitations and next steps

The system is a complete simulation-based prototype, not yet connected to real robots.

Honest limitations

- All results come from the simulated digital twin; the separate-managers scenario rests on stated assumptions (90 s operator response, 20 s WMS hand-over delay, dedicated chargers per vendor) that should be checked with real sites.
- The vendor endpoints are simulated. The adapters produce real protocol messages, and VDA 5050 traffic goes over a real MQTT broker, but no physical robot controller has been connected.
- Throughput is equal in both scenarios whenever orders arrive slower than the fleet can handle them. TusCore's gain then shows as faster, smoother work and no manual intervention, not as more orders.
- On the large layout (3 runs only) TusCore's average wait came out 9% higher than with separate managers, while cycle time was 11% lower. This is reported as measured.
- The Docker image was not build-tested in the development environment, and the logos in the console are placeholders until the official ones are supplied.

Next steps toward a live site

1. Connect the adapters to real controllers (VDA 5050 broker, vendor REST endpoints, TCP gateway, drone link) and drive the twin from live telemetry.
2. Import the real site map into the layout file and align each vendor's coordinate frame.
3. Re-run the comparison and what-if with the site's real order profile.
4. Production hardening: login and roles, audit log, high availability, TLS on the broker and API.
5. Optional stretch goals from the brief: a ROS 2 / Gazebo bridge for one robot, natural-language order entry, and a MOV.AI maturity-model assessment.

Appendix

How to run it

- **Docker (one command, includes the Mosquitto broker):** `docker compose up --build`, then open http://localhost:8000
- **Windows without Docker:** double-click `run.bat`, then open http://localhost:8000
- **API documentation:** http://localhost:8000/docs
- **Tests:** in the `backend` folder, `pip install -r requirements-dev.txt` and `python -m pytest -q`
- **One shift without the browser:** `python scripts/run_shift.py --mode separate` (about 6 s for 8 hours)
- **Layouts:** `config/warehouse.yaml` (60x40, 12 robots), `warehouse_large.yaml` (80x50, 19 robots), `warehouse_xl.yaml` (100x64, 40 robots)

Demo video script (about 4 minutes)

Time	Show	Say
0:00	Live page, 3D twin at 10x, orbit the warehouse	"Twelve robots, four vendors, four protocols, one TusCore."
0:30	Follow the scissor lift during a high-shelf pick and hand-over	"Two robot types from two vendors, coordinated on one order."
1:00	Protocol tab	"The same core speaks VDA 5050, REST, TCP/JSON and MAVLink."
1:20	Inject a fault on a busy robot, open Events	"The robot fails, its order is reassigned automatically."
1:50	Close an aisle for maintenance, switch on the heatmap	"Traffic reroutes; the heatmap shows where robots lose time."
2:20	Click the mode pill to show separate managers	"This is today's typical site: congestion and manual rescues."
2:50	Compare page, then What-if with two extra tote robots	"22% faster orders, 79% less traffic delay, zero interventions."
3:40	Overview page, then the API docs	"Any WMS can plug in through the REST API."

Glossary

Term	Meaning
AMR / AGV	Autonomous mobile robot / automated guided vehicle
Digital twin	A live virtual copy of the warehouse and its robots
VDA 5050	Open standard for communication between a fleet manager and mobile robots, usually over MQTT
MQTT / Mosquitto	Lightweight publish/subscribe messaging protocol / a widely used open-source MQTT broker
MAVLink	Messaging protocol used by drones
A*	Shortest-path search algorithm used for route planning
Hungarian algorithm	Method that finds the best overall matching of tasks to robots
Deadlock	Robots waiting for each other in a loop, so none can move
Cycle time	Time from an order arriving to it being completed
Utilization	Share of time robots are busy (moving, working or waiting at a hand-over)